

Invariantes: acumuladores, listas y ciclos anidados

Demostrar un ciclo cuando el estado crece

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre de 2026

Objetivos de la sesión

Al terminar la clase, usted podrá

- Escribir invariantes para un ciclo cuyo estado son varias variables que se actualizan juntas (OA6).
- Escribir el invariante de un ciclo que construye una lista, diciendo cuántos elementos lleva y qué hay en cada uno (OA6).
- Demostrar un ciclo anidado en dos etapas: el interno como lema, el externo sobre la conclusión del interno (OA6).
- Reconocer las variantes de la búsqueda binaria como el mismo algoritmo sobre predicados distintos (OA1).

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes
- 7 Ejercicios
- 8 Referencias

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes
- 7 Ejercicios
- 8 Referencias

Lo que promete un invariante

La pareja de siempre

- I_0 **acota el índice**: dice por dónde puede ir.
- I_1 describe el **estado**: dice qué lleva calculado, en función del índice.

Y la demostración va en tres pasos: **inicialización** (valen antes de la primera vuelta), **estabilidad** (una vuelta cualquiera los conserva) y **terminación** (el ciclo para, y ahí los invariantes entregan la poscondición).

El paso que se olvida

En la terminación el valor final del índice no se declara: sale de **interceptar la condición rota con I_0** . Si el ciclo se detuvo, $i < N$ es falsa, o sea $i \geq N$; junto con $I_0 : 0 \leq i \leq N$ eso da $i = N$, y no otra cosa.

Dónde quedó el método

Lo que ya está demostrado

La suma de un arreglo, el factorial, la búsqueda de un valor, y la correctitud de la búsqueda binaria. En los cuatro el estado del ciclo era lo mismo: **un índice y un acumulador numérico**, y I_1 era una ecuación entre los dos.

Lo que falta

Tres situaciones donde ese molde numérico no alcanza:

- 1 El estado son **varias variables** que se actualizan en la misma vuelta y dependen unas de otras.
- 2 La salida es una **lista que crece**, no un número.
- 3 Hay **un ciclo dentro de otro**, y el externo depende de lo que garantice el interno.

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes
- 7 Ejercicios
- 8 Referencias

El n -ésimo número de Fibonacci

Definición (Sucesión de Fibonacci, CLRS Ecuación 3.21, p. 59)

$$F(0) = 0, \quad F(1) = 1, \quad F(k) = F(k-1) + F(k-2) \text{ para } k \geq 2.$$

Dos caminos

Escribir esa definición como función recursiva funciona, pero recalcula: $F(5)$ pide $F(4)$ y $F(3)$, y $F(4)$ vuelve a pedir $F(3)$. El árbol de llamadas crece de forma exponencial.

Con un ciclo basta cargar **los dos últimos valores** y correrlos una posición en cada vuelta. El costo baja a $\Theta(n)$, pero ahora el estado del ciclo son dos números amarrados entre sí.

La implementación

```
1 def fibonacci(n):  
2     a = 0  
3     b = 1  
4     i = 0  
5     while i < n:  
6         siguiente = a + b  
7         a = b  
8         b = siguiente  
9         i = i + 1  
0     return a
```

La variable siguiente no sobra

La línea 7 destruye el valor viejo de a. Si la línea 8 tuviera que volver a sumarlo, ya no lo encontraría. Guardar la suma antes de mover nada es lo que deja las dos asignaciones independientes del orden en que se lean.

Los invariantes

Un invariante puede hablar de varias variables

$$I_0 : 0 \leq i \leq n \qquad I_1 : a = F(i) \wedge b = F(i + 1)$$

Cómo se lee

I_1 no es una ecuación sino una conjunción: fija de un golpe qué guarda cada variable, y las dos referidas al mismo índice i . Esa es la forma general; el caso de un solo acumulador es la conjunción de un término.

Por qué no alcanza con $a = F(i)$

Con solo esa mitad, la estabilidad se atasca: para calcular el nuevo valor de a hace falta saber qué había en b , y el invariante no lo dice. El estado que el ciclo arrastra tiene que quedar descrito **completo**.

Fibonacci: Teorema 1 e inicialización

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Inicialización

De acuerdo a las líneas 2–4, inicialmente $a = 0$, $b = 1$ e $i = 0$. Para I_0 : $0 \leq 0 \leq n$, cierto porque la especificación pide $n \geq 0$. ✓ Para I_1 : $a = 0 = F(0) = F(i)$ y $b = 1 = F(1) = F(i + 1)$, ambos por definición de la sucesión. ✓ Por lo tanto, los invariantes se cumplen antes de la primera iteración.

Fibonacci: la estabilidad

Estabilidad

Se considera una iteración arbitraria $i = j$ que ejecuta el cuerpo, o sea con $j < n$, y se asume que antes de ella valen $0 \leq j \leq n$, $a = F(j)$ y $b = F(j + 1)$. Al ejecutar las líneas 6–9, y nombrando a' , b' e i' a los valores nuevos:

$$\text{siguiente} = a + b = F(j) + F(j + 1) = F(j + 2),$$

$$a' = b = F(j + 1),$$

$$b' = \text{siguiente} = F(j + 2) = F((j + 1) + 1),$$

$$i' = j + 1.$$

Fibonacci: la estabilidad

La tercera igualdad de la primera línea es la definición de F aplicada a $k = j + 2$, que exige $j + 2 \geq 2$ y se cumple porque $j \geq 0$. Entonces $a' = F(i')$ y $b' = F(i' + 1)$, que es I_1 evaluado en $j + 1$. ✓ Para I_0 : de $j < n$ sale $j + 1 \leq n$, y de $j \geq 0$ sale $j + 1 \geq 0$, luego $0 \leq j + 1 \leq n$. ✓ Por lo tanto, los invariantes son estables.

Fibonacci: terminación y teorema del algoritmo

Terminación

El ciclo termina porque i crece de 1 en 1 desde 0 e l_0 lo acota por n . Al salir, la condición $i < n$ es falsa, o sea $i \geq n$; junto con l_0 eso da $i = n$. Sustituyendo en l_1 : $a = F(n)$. ■

Teorema 2

La invocación `fibonacci(n)` para cualquier entero $n \geq 0$ produce como resultado $F(n)$.

Demostración: se procede de forma directa a partir del Teorema 1. La terminación deja $i = n$, y el primer término de l_1 evaluado en n dice $a = F(n)$, que es justamente lo que retorna la línea 10. ■

Cuando el orden de las asignaciones traiciona

Una versión que parece igual

$$a = b$$

$$b = a + b$$

La línea 1 deja $a = F(j + 1)$, correcto. Pero la línea 2 ya no encuentra el a viejo: calcula $b' = F(j + 1) + F(j + 1) = 2F(j + 1)$, y para que I_1 valga en $j + 1$ haría falta $F(j + 2) = F(j + 1) + F(j)$. Las dos coinciden solo si $F(j) = F(j + 1)$, que pasa únicamente en $j = 1$.

Lo que hay que ver

El error no se descubre corriendo el programa con $n = 2$, donde por casualidad acierta. Se descubre en la estabilidad, que es donde el argumento no cierra.

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista**
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes
- 7 Ejercicios
- 8 Referencias

¿Qué tiene que decir el invariante de una lista?

Dos cosas, no una

Cuando el ciclo devuelve una lista que fue creciendo, el invariante tiene que fijar:

- 1 **Cuántos elementos lleva**, en función del índice.
- 2 **Qué hay en cada posición ya escrita**, con una fórmula o una descripción.

Si falta la primera

En la terminación se sabría qué hay en cada posición de res, pero no cuántas posiciones hay. Una lista vacía cumpliría “todo lo que está escrito está bien” de forma trivial, y la poscondición no saldría.

¿Qué tiene que decir el invariante de una lista?

Y no siempre es $\text{len}(\text{res}) = i$

Si la lista crece en todas las vueltas, sí. Si crece solo cuando se cumple una condición, el largo va por su cuenta y el invariante tiene que decir de qué depende.

Primer caso: la lista crece a veces

```
1 def posiciones_pares(A):  
2     N = len(A)  
3     res = []  
4     i = 0  
5     while i < N:  
6         if A[i] % 2 == 0:  
7             res.append(i)  
8             i = i + 1  
9     return res
```

Primer caso: la lista crece a veces

Los invariantes

$$I_0 : 0 \leq i \leq N$$

I_1 : res es la lista, en orden creciente, de las posiciones j con $0 \leq j < i$ y $A[j]$ par.

El largo no aparece: aquí I_1 ya lo determina, porque describe *exactamente* qué posiciones están y en qué orden.

posiciones_pares: Teorema 1

Teorema 1

Los invariantes I_0 e I_1 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Inicialización

Por las líneas 3–4, $res = []$ e $i = 0$. Para I_0 : $0 \leq 0 \leq N$, cierto porque $N = \text{len}(A) \geq 0$. ✓ Para I_1 : el rango $0 \leq j < 0$ es vacío, así que la lista de sus posiciones pares es la lista vacía, que es lo que hay en res . ✓

posiciones_pares: estabilidad

Estabilidad

Se considera una iteración arbitraria $i = j$ con $j < N$, y se asume que res contiene, en orden creciente, las posiciones pares de $A[0..j)$. Hay dos casos según la línea 6.

Si $A[j]$ es par, la línea 7 agrega j al final. Como todas las posiciones que ya estaban son menores que j , agregar al final conserva el orden creciente, y la lista pasa a contener las posiciones pares de $A[0..j + 1)$. ✓

Si $A[j]$ es impar, res no cambia. La posición j no debe estar, y las de $A[0..j)$ ya están, así que la lista contiene las posiciones pares de $A[0..j + 1)$. ✓

En ambos casos vale I_1 evaluado en $j + 1$. Para I_0 : de $j < N$ sale $0 \leq j + 1 \leq N$. ✓ Por lo tanto, los invariantes son estables.

posiciones_pares: terminación y teorema

Terminación

El ciclo termina porque i crece de 1 en 1 e l_0 lo acota por N . Al salir, $i < N$ es falsa, o sea $i \geq N$; con l_0 eso da $i = N$. Sustituyendo en l_1 : res es la lista creciente de las posiciones $j < N$ con $A[j]$ par, o sea todas las del arreglo. ■

Teorema 2

La invocación `posiciones_pares(A)` para cualquier arreglo de enteros $A[0..N)$ con $N \geq 0$ produce la lista creciente de las posiciones donde A tiene un valor par.

Demostración: se procede de forma directa a partir del Teorema 1, sustituyendo $i = N$ en l_1 , que es lo que retorna la línea 9. ■

Segundo caso: la lista crece siempre

El problema

Dado $A[0..N)$ con $N \geq 1$, devolver la lista $R[0..N)$ donde $R[j]$ es el mayor valor entre $A[0]$ y $A[j]$, ambos incluidos. Con $A = [4, 2, 9, 9, 1]$ la respuesta es $[4, 4, 9, 9, 9]$.

```
1 def maximos_parciales(A):  
2     N = len(A)  
3     m = A[0]  
4     res = [A[0]]  
5     i = 1  
6     while i < N:  
7         if A[i] > m:  
8             m = A[i]  
9             res.append(m)  
10            i = i + 1  
11    return res
```


Tres invariantes, uno por pieza del estado

El estado tiene tres partes, y cada una necesita su línea

$$l_0 : 1 \leq i \leq N$$

$$l_1 : m = \text{máx } A[0..i)$$

$$l_2 : \text{len}(\text{res}) = i \wedge \forall j, 0 \leq j < i : \text{res}[j] = \text{máx } A[0..j]$$

Por qué el índice arranca en 1

Las líneas 3–5 ya resolvieron la posición 0. Si el ciclo empezara en $i = 0$, l_1 pediría el máximo del rango vacío $A[0..0)$, que no existe. Mover el arranque a 1 elimina el caso especial en vez de tener que arrastrarlo.

Tres invariantes, uno por pieza del estado

La precondition no es adorno

Con $N = 0$ la línea 3 falla antes de llegar al ciclo. La especificación pide $N \geq 1$ y el código lo aprovecha.

maximos_parciales: Teorema 1 e inicialización

Teorema 1

Los invariantes I_0 , I_1 e I_2 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación.

Inicialización

Por las líneas 3–5, $m = A[0]$, $res = [A[0]]$ e $i = 1$. Para I_0 : $1 \leq 1 \leq N$, cierto por la precondition $N \geq 1$. ✓ Para I_1 : $\max A[0..1) = A[0] = m$. ✓ Para I_2 : $\text{len}(res) = 1 = i$, y el único j con $0 \leq j < 1$ es $j = 0$, donde $res[0] = A[0] = \max A[0..0]$. ✓

maximos_parciales: la estabilidad

Estabilidad

Se considera una iteración arbitraria $i = j$ con $j < N$, y se asumen $1 \leq j \leq N$, $m = \max A[0..j]$, $\text{len}(\text{res}) = j$ y $\text{res}[k] = \max A[0..k]$ para todo $k < j$.

Las líneas 7–8. Si $A[j] > m$ el nuevo valor es $m' = A[j]$; si no, $m' = m$. En los dos casos $m' = \max\{m, A[j]\}$, y sustituyendo m por lo que promete I_1 :

$$m' = \max \{ \max A[0..j), A[j] \} = \max A[0..j] = \max A[0..j + 1),$$

que es I_1 evaluado en $j + 1$. ✓

maximos_parciales: la estabilidad

La línea 9. El append deja $\text{len}(\text{res}) = j + 1$. ✓ Escribe en la posición j el valor $m' = \max A[0..j]$, que es lo que I_2 pide para $k = j$; y no toca las posiciones $k < j$, que seguían valiendo por hipótesis. ✓

Para I_0 : de $j < N$ sale $1 \leq j + 1 \leq N$. ✓ Por lo tanto, los tres invariantes son estables.

maximos_parciales: terminación y teorema

Terminación

El ciclo termina porque i crece de 1 en 1 desde 1 e l_0 lo acota por N . Al salir, $i < N$ es falsa, o sea $i \geq N$; con l_0 eso da $i = N$. Sustituyendo en l_2 : $\text{len}(\text{res}) = N$ y $\text{res}[j] = \text{máx } A[0..j]$ para todo $j < N$, que es la poscondición completa: el largo y el contenido. ■

Teorema 2

La invocación `maximos_parciales(A)` para cualquier arreglo $A[0..N)$ con $N \geq 1$ produce la lista $R[0..N)$ con $R[j] = \text{máx } A[0..j]$.

Demostración: se procede de forma directa a partir del Teorema 1, sustituyendo $i = N$ en l_2 , que describe exactamente lo que retorna la línea 11. ■

maximos_parciales: terminación y teorema

Note que I_1 no aparece en el teorema

m es una variable de trabajo: su invariante existe para que la estabilidad de I_2 pueda apoyarse en él. No todo invariante termina en la poscondición; algunos solo sostienen a otros.

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados**
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes
- 7 Ejercicios
- 8 Referencias

Dos ciclos, dos demostraciones

El orden del trabajo

- 1 Se escriben los invariantes del **ciclo interno**, con el índice del externo *fijo*, y se demuestran con los tres pasos. La conclusión se guarda como un **lema**.
- 2 Se escriben los invariantes del **ciclo externo**. En su estabilidad, el ciclo interno entero se resume en una línea: la del lema.

El error que hay que evitar

Volver a abrir el ciclo interno dentro de la estabilidad del externo. Si el lema ya está demostrado, el externo lo **cita**: “por el Lema 1, al salir del ciclo interno vale tal cosa”. Si no se cita, la demostración del externo se convierte en la del interno otra vez, y no cierra.

Dos ciclos, dos demostraciones

El invariante del interno lleva el índice del externo

Escribir el invariante interno sin decir que i está fijo lo deja sin sentido: i es un parámetro del lema, no una variable que se mueva.

Ordenamiento por selección

La idea

Buscar el menor de todo el arreglo y ponerlo de primero. Buscar el menor de lo que queda y ponerlo de segundo. Repetir. Al llegar al último elemento no hay nada que buscar: ya es el mayor.

```
1 def ordenar_por_seleccion(A):
2     N = len(A)
3     i = 0
4     while i < N - 1:
5         p = i
6         j = i + 1
7         while j < N:
8             if A[j] < A[p]:
9                 p = j
10            j = j + 1
11        temporal = A[i]
12        A[i] = A[p]
13        A[p] = temporal
14        i = i + 1
15    return A
```

La traza sobre $[5, 2, 9, 1, 6]$

i	p al salir del interno	intercambio	A después
0	3	$A[0] \leftrightarrow A[3]$	$[1, 2, 9, 5, 6]$
1	1	$A[1] \leftrightarrow A[1]$	$[1, 2, 9, 5, 6]$
2	3	$A[2] \leftrightarrow A[3]$	$[1, 2, 5, 9, 6]$
3	4	$A[3] \leftrightarrow A[4]$	$[1, 2, 5, 6, 9]$

Lo que muestra la columna de la derecha

El prefijo en **negrita** crece de a uno y nunca se vuelve a tocar. El último elemento queda en su sitio sin que nadie lo mueva: cuando $i = N - 1$ el ciclo ya paró.

El ciclo interno: qué promete

Con i fijo, sobre las líneas 5–10

$$J_0 : i + 1 \leq j \leq N \wedge i \leq p < j$$

$$J_1 : A[p] = \text{mín } A[i..j]$$

J_1 dice que p marca la posición de un mínimo de la parte ya revisada. J_0 acota los dos índices: j avanza dentro del arreglo y p siempre apunta a algo ya visto.

Lema (El ciclo interno encuentra el mínimo)

Si al entrar al cuerpo del ciclo externo vale $0 \leq i < N - 1$, entonces al salir del ciclo interno se cumple $A[p] = \text{mín } A[i..N)$ con $i \leq p < N$.

Lema 1: inicialización

Inicialización

Por las líneas 5–6, $p = i$ y $j = i + 1$.

Para J_0 : la primera parte es $i + 1 \leq i + 1 \leq N$, y $i + 1 \leq N$ sale de la condición del externo $i < N - 1$, que da $i \leq N - 2$. ✓ La segunda es $i \leq i < i + 1$. ✓

Para J_1 : el rango $A[i..i + 1)$ tiene un solo elemento, $A[i]$, así que su mínimo es $A[i] = A[p]$. ✓

Por lo tanto, los invariantes J_0 y J_1 se cumplen antes de la primera iteración del ciclo interno.

Lema 1: la estabilidad

Estabilidad

Se considera una iteración arbitraria $j = q$ que ejecuta el cuerpo, o sea con $q < N$, y se asume $A[p] = \min A[i..q]$ con $i \leq p < q$. Sea p' el valor de p después de las líneas 8–9. Hay dos casos.

Si $A[q] < A[p]$, la línea 9 deja $p' = q$. Entonces $A[p'] = A[q]$, y como $A[q] < A[p] = \min A[i..q]$, ese valor es menor que todos los de $A[i..q]$; por lo tanto es el mínimo de $A[i..q] \cup \{A[q]\} = A[i..q+1]$. ✓ Además $i \leq q < q+1$. ✓

Si $A[q] \geq A[p]$, p no cambia, o sea $p' = p$. Entonces $A[p']$ sigue siendo menor o igual que todos los de $A[i..q]$ por hipótesis, y también menor o igual que $A[q]$ por la condición de este caso; luego es el mínimo de $A[i..q+1]$. ✓ Y $i \leq p < q < q+1$. ✓

En ambos casos vale J_1 evaluado en $q+1$. Para el resto de J_0 : de $q < N$ sale $i+1 \leq q+1 \leq N$. ✓ Por lo tanto, los invariantes son estables.

Lema 1: la terminación

Terminación

El ciclo interno termina porque j crece de 1 en 1 desde $i + 1$ y J_0 lo acota por N . Al salir, la condición $j < N$ es falsa, o sea $j \geq N$; junto con J_0 eso da $j = N$. Sustituyendo ese valor en J_1 :

$$A[p] = \text{mín } A[i..N),$$

y de J_0 queda $i \leq p < N$. Por lo tanto, el Lema 1 es correcto. ■

Lo que el externo va a usar

Solo esas dos afirmaciones. Cómo las consiguió el ciclo interno deja de importar a partir de aquí.

El ciclo externo: los invariantes

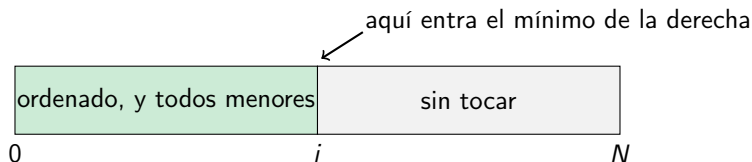
Sobre las líneas 3–14

$I_0 : 0 \leq i \leq N - 1$

$I_1 : A[0..i)$ está en orden no decreciente

$I_2 : \text{todo elemento de } A[0..i) \text{ es } \leq \text{todo elemento de } A[i..N)$

$I_3 : A \text{ es una permutación del arreglo original}$



¿Por qué hace falta I_2 ?

Con I_1 solo no cierra

Saber que $A[0..i)$ está ordenado no dice nada sobre lo que viene después. El arreglo $[1, 2, | 0, 7]$ cumple I_1 con $i = 2$ y no está en camino de quedar ordenado: el 0 de la derecha debería haber ido de primero.

Lo que aporta cada uno

I_1 ordena hacia adentro del prefijo. I_2 lo separa del resto: garantiza que nada de la derecha tiene que volver a la izquierda. Los dos juntos son los que hacen que en la terminación el arreglo entero quede ordenado.

Y I_3

Ordenar no es solo entregar algo ordenado: es entregar *los mismos elementos* ordenados. Un ciclo que llenara el arreglo de ceros cumpliría I_1 e I_2 sin ser un ordenamiento.

Teorema 1 e inicialización

Teorema 1

Los invariantes I_0 , I_1 , I_2 e I_3 se cumplen.

Demostración: se procede mostrando la validez de los invariantes para la inicialización, la estabilidad y la terminación, usando el Lema 1 en la estabilidad.

Inicialización

Por la línea 3, $i = 0$. Para I_0 : $0 \leq 0 \leq N - 1$, cierto siempre que $N \geq 1$; con $N = 0$ el ciclo no corre y no hay nada que probar. ✓ Para I_1 : $A[0..0)$ es vacío y un rango vacío está ordenado. ✓ Para I_2 : la afirmación cuantifica sobre un rango vacío, luego es cierta de forma trivial. ✓ Para I_3 : todavía no se ha modificado nada. ✓

La estabilidad, apoyada en el Lema 1

Estabilidad

Se considera una iteración arbitraria $i = k$ que ejecuta el cuerpo, o sea con $k < N - 1$, y se asumen I_0 a I_3 para k . Por el **Lema 1**, al salir del ciclo interno vale $A[p] = \min A[k..N)$ con $k \leq p < N$. Las líneas 11–13 intercambian $A[k]$ con $A[p]$; sea B el arreglo después del intercambio.

I_3 . El intercambio permuta dos posiciones, así que B sigue siendo una permutación del original. ✓

I_2 **sobrevivía al intercambio**. Las dos posiciones tocadas, k y p , están ambas en $[k..N)$. El multiconjunto $A[0..k)$ no cambió y el multiconjunto $A[k..N)$ tampoco: sus elementos solo se reacomodaron. Luego I_2 sigue valiendo entre $B[0..k)$ y $B[k..N)$.

La estabilidad, apoyada en el Lema 1

I_1 en $k + 1$. $B[0..k) = A[0..k)$ está ordenado por hipótesis. Falta ver que $B[k]$ no rompe el orden al final: $B[k] \in A[k..N)$, y por I_2 todo elemento de $A[0..k)$ es $\leq$ todo elemento de $A[k..N)$, en particular $\leq B[k]$. Luego $B[0..k + 1)$ está ordenado. ✓

I_2 en $k + 1$. Para los elementos de $B[0..k)$ vale por lo ya dicho, porque $B[k + 1..N) \subseteq B[k..N)$. Para $B[k]$: es mín $A[k..N)$ por el Lema 1, y $B[k + 1..N)$ son elementos de ese mismo rango, luego $B[k] \leq$ todos ellos. ✓

I_0 . De $k < N - 1$ sale $0 \leq k + 1 \leq N - 1$. ✓ Por lo tanto, los cuatro invariantes son estables.

Terminación y teorema del algoritmo

Terminación

El ciclo externo termina porque i crece de 1 en 1 desde 0 e I_0 lo acota por $N - 1$. Al salir, la condición $i < N - 1$ es falsa, o sea $i \geq N - 1$; junto con I_0 eso da $i = N - 1$. Sustituyendo:

- I_1 dice que $A[0..N - 1)$ está ordenado.
- I_2 dice que todo elemento de $A[0..N - 1)$ es $\leq$ el único elemento de $A[N - 1..N)$, que es $A[N - 1]$.

Un arreglo ordenado al que se le agrega al final un elemento mayor o igual que todos sigue ordenado, luego $A[0..N)$ está ordenado. Con I_3 , además, son los elementos originales. ■

Terminación y teorema del algoritmo

Teorema 2

La invocación `ordenar_por_seleccion(A)` para cualquier arreglo $A[0..N)$ con $N \geq 0$ devuelve una permutación de A en orden no decreciente.

Demostración: se procede de forma directa a partir del Teorema 1. Con $N \leq 1$ el ciclo no corre y el arreglo ya cumple la poscondición; con $N \geq 2$ la terminación la entrega. ■

¿Cuánto cuesta?

Contando las vueltas del interno

Para cada i , el ciclo interno recorre $A[i + 1..N)$, o sea $N - i - 1$ comparaciones. Sumando sobre todos los valores de i :

$$\sum_{i=0}^{N-2} (N - i - 1) = \sum_{t=1}^{N-1} t = \frac{N(N-1)}{2}.$$

Ese conteo no depende del contenido del arreglo: el interno recorre todo el sufijo aunque el mínimo aparezca de primero. Peor caso y mejor caso coinciden en $\Theta(N^2)$.

¿Cuánto cuesta?

Frente al ordenamiento por mezcla

Mezcla cuesta $\Theta(N \lg N)$ y este $\Theta(N^2)$. Con $N = 10^6$ la diferencia es de unas 25 000 veces. Lo que selección tiene a favor es que ordena sobre el mismo arreglo, sin pedir memoria extra, y que hace a lo sumo $N - 1$ intercambios.

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria**
- 6 Errores comunes
- 7 Ejercicios
- 8 Referencias

Las preguntas que la versión de siempre no responde

Sobre $A = [1, 3, 3, 3, 7, 9]$

- ¿Dónde *empieza* el bloque de treses? ¿Dónde termina?
- ¿Cuántas veces aparece el 3?
- El 5 no está. ¿Cuál es el mayor elemento que no lo supera?
- ¿En qué posición habría que insertar el 5 para conservar el orden?

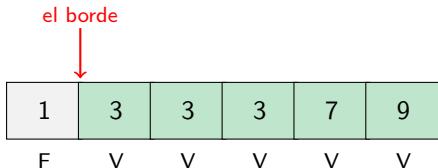
Son la misma búsqueda

La versión que devuelve “la posición de x , o -1 ” contesta ninguna de las cuatro: con repetidos entrega *alguna* posición, sin decir cuál, y cuando x no está tira el trabajo hecho. Las cuatro salen del mismo ciclo cambiando una comparación.

Buscar un borde, no un elemento

El predicado y su monotonía

Sobre un arreglo ordenado, el predicado $P(j) : A[j] \geq x$ recorre las posiciones así:



Buscar un borde, no un elemento

Por qué sirve

El orden del arreglo obliga a que los falsos vayan todos primero y los verdaderos todos después. Ese es el mismo terreno de la bisección: una función monótona y un borde que encontrar partiendo el intervalo. Cambiar la pregunta es cambiar el predicado, no el algoritmo.

El ciclo que encuentra el borde

```
1 def primera_posicion_mayor_o_igual(A, x):
2     l = 0
3     r = len(A)
4     while l < r:
5         mitad = (l + r) // 2
6         if A[mitad] < x:
7             l = mitad + 1
8         else:
9             r = mitad
10    return l
```

El ciclo que encuentra el borde

Los invariantes

$$I_0 : 0 \leq l \leq r \leq N$$

$$I_1 : \forall j < l : A[j] < x$$

$$I_2 : \forall j \geq r : A[j] \geq x$$

Al terminar, $l = r$, y las dos afirmaciones se tocan: todo lo anterior a l es menor que x y todo lo que sigue desde l no lo es. Eso es exactamente el borde, exista x o no.

Tres detalles que cuestan puntos

No hay mitad - 1

En la línea 9 se escribe $r = \text{mitad}$, no $\text{mitad} - 1$. La posición mitad cumple el predicado y podría ser el borde: descartarla es perderlo.

El intervalo es medio abierto

r arranca en $\text{len}(A)$, no en $\text{len}(A) - 1$. Ese valor de más es el que permite responder “ninguna posición cumple” devolviendo N , en lugar de tener que inventar un caso especial.

La condición es $l < r$

Con $l \leq r$ el ciclo no termina: cuando $l = r$ se tiene $\text{mitad} = l$, y la rama del `else` vuelve a poner $r = l$ sin mover nada.

Las otras tres preguntas, gratis

```
def primera_posicion_mayor(A, x):  
    l = 0  
    r = len(A)  
    while l < r:  
        mitad = (l + r) // 2  
        if A[mitad] <= x:  
            l = mitad + 1  
        else:  
            r = mitad  
    return l
```

Lo único que cambió

El $<$ de la comparación pasó a $\leq$. El predicado ahora es $A[j] > x$, y el borde se corre al *final* del bloque de copias de x .

Y con los dos bordes se responde todo

```
def esta(A, x):  
    p = primera_posicion_mayor_o_igual(A, x)  
    return p < len(A) and A[p] == x
```

```
def cuantas_veces(A, x):  
    return primera_posicion_mayor(A, x) -  
        ↪ primera_posicion_mayor_o_igual(A, x)
```

```
def mayor_menor_o_igual(A, x):  
    return primera_posicion_mayor(A, x) - 1
```

Y con los dos bordes se responde todo

Sobre $A = [1, 3, 3, 3, 7, 9]$

Los dos bordes del 3 son 1 y 4: el bloque ocupa las posiciones 1, 2 y 3, y $4 - 1 = 3$ copias. Para $x = 5$, `primera_posicion_mayor` devuelve 4, y $4 - 1 = 3$ es la posición del 3, el mayor elemento que no supera al 5.

Cuando todos son mayores que x , el borde vale 0 y la resta devuelve -1 , que es la señal de “no hay”.

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes**
- 7 Ejercicios
- 8 Referencias

Lo que sale mal

En el estado con varias variables

- Describir solo una parte del estado. Si la estabilidad necesita un valor que ningún invariante menciona, el argumento se atasca.
- Usar el valor nuevo de una variable donde el código todavía lee el viejo. En la estabilidad conviene nombrarlos aparte: a y a' .

En la lista que crece

- Decir qué hay en cada posición y no cuántas posiciones hay. La terminación entrega media poscondición.
- Suponer $\text{len}(\text{res}) = i$ cuando la lista crece bajo condición.

Lo que sale mal

En los ciclos anidados

- Escribir el invariante del interno sin fijar el índice del externo.
- Repetir la demostración del interno dentro de la estabilidad del externo, en vez de citar el lema.
- Olvidar que el cuerpo del externo modifica el arreglo, y dar por hecho que los invariantes valen sobre el arreglo de antes.

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes
- 7 Ejercicios**
- 8 Referencias

Para practicar el método

Con un ciclo

- 1 Los récords.** Construya la lista de las posiciones j donde $A[j]$ es mayor que todos los anteriores. Necesita un acumulador y una lista: escriba un invariante para cada uno y demuestre los tres pasos.
- 2 Invertir un número.** Dado $n \geq 0$, devuelva el número con sus dígitos al revés. El dato del parámetro se consume en cada vuelta; el invariante tiene que relacionar lo consumido con lo construido.

Para practicar el método

Con dos ciclos

- 3 **Ordenamiento por inserción** (CLRS, Sección 2.1, p. 18). El ciclo interno corre elementos hacia la derecha en lugar de solo mirar: su invariante habla de dónde quedó cada uno. Demuéstrelo como lema y después escriba el externo sobre esa conclusión.
- 4 **Contar inversiones**. Cuente los pares (a, b) con $a < b$ y $A[a] > A[b]$, con dos ciclos. El invariante del interno cuenta las inversiones que empiezan en a ; el del externo acumula las de $A[0..a]$.

Para practicar el método

Con búsqueda binaria

- 5 Escriba `mayor_menor_o_igual` con su propio ciclo, sin llamar a las otras dos, y dé sus invariantes.
- 6 ¿Qué devuelve `primera_posicion_mayor_o_igual` sobre un arreglo vacío? Sigálo con los invariantes en la mano.

Estos ejercicios entran en el material del parcial.

Contenido

- 1 Repaso: el método
- 2 Varias variables a la vez
- 3 Cuando la salida es una lista
- 4 Ciclos anidados
- 5 Variantes de la búsqueda binaria
- 6 Errores comunes
- 7 Ejercicios
- 8 Referencias**

Referencias

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, MIT Press, 2009. Sección 2.1 (invariantes de ciclo, pp. 18–20), Ejercicio 2.2-2 (ordenamiento por selección, p. 29) y Ecuación 3.21 (Fibonacci, p. 59).
-  C. Rocha. *Diseño y Análisis de Algoritmos*.
-  J. Kleinberg y É. Tardos. *Algorithm Design*, Addison-Wesley, 2005.
-  S. Halim, F. Halim y S. Effendy. *Competitive Programming 4*, 2018. Capítulo 2: las variantes de la búsqueda binaria.